

Programming AI

BS(AI) Part-2- 2nd Semester

Prof. Dr. Imtiaz Ali Korejo

Python Data Types

Week-2 , Lecture-4





Outline of Today's Lesson

1. Python Data Types

1. Setting the Data Type
2. Python Numbers
3. Complex Numbers,
4. Random Numbers
5. Python Casting

2. Python Strings

1. Quotes
2. Multiline String
3. Strings are Arrays
4. Looping through String

5. String Length

6. Check string

7. Cjeck if Not



Section-1

Python Programming

PYTHON DATA TYPES



By: Dr. Imtiaz Ali Korejo



Python Data Types

- In programming, data type is an important concept.
- Variables can store data of different types, and different types can do different things.
- Python has the following data types built-in by default, in these categories:

Text Type:	str
Numeric Types:	int, float, complex
Sequence Types:	list, tuple, range
Mapping Type:	dict
Set Types:	set, frozenset
Boolean Type:	bool
Binary Types:	bytes, bytearray, memoryview
None Type:	NoneType





Getting the Data Type

- You can get the data type of any object by using the type() function:

```
x = 5  
print(type(x))
```

```
Output:  
<class 'int'>
```





Setting the Data Type

- In Python, the data type is set when you assign a value to a variable:

<code>x = "Hello World"</code>	<code>str</code>	
<code>x = 20</code>	<code>int</code>	
<code>x = 20.5</code>	<code>float</code>	
<code>x = 1j</code>	<code>complex</code>	
<code>x = ["apple", "banana", "cherry"]</code>	<code>list</code>	
<code>x = ("apple", "banana", "cherry")</code>	<code>tuple</code>	
<code>x = range(6)</code>	<code>range</code>	
<code>x = {"name" : "John", "age" : 36}</code>	<code>dict</code>	
<code>x = {"apple", "banana", "cherry"}</code>	<code>set</code>	
<code>x = frozenset({"apple", "banana", "cherry"})</code>	<code>frozenset</code>	
<code>x = True</code>	<code>bool</code>	
<code>x = b"Hello"</code>	<code>bytes</code>	
<code>x = bytearray(5)</code>	<code>bytearray</code>	
<code>x = memoryview(bytes(5))</code>	<code>memoryview</code>	
<code>x = None</code>	<code>NoneType</code>	





Setting the Specific Data Type

- If you want to specify the data type, you can use the following constructor functions:

<code>x = str("Hello World")</code>	<code>str</code>
<code>x = int(20)</code>	<code>int</code>
<code>x = float(20.5)</code>	<code>float</code>
<code>x = complex(1j)</code>	<code>complex</code>
<code>x = list(("apple", "banana", "cherry"))</code>	<code>list</code>
<code>x = tuple(("apple", "banana", "cherry"))</code>	<code>tuple</code>
<code>x = range(6)</code>	<code>range</code>
<code>x = dict(name="John", age=36)</code>	<code>dict</code>
<code>x = set(("apple", "banana", "cherry"))</code>	<code>set</code>
<code>x = frozenset(("apple", "banana", "cherry"))</code>	<code>frozenset</code>
<code>x = bool(5)</code>	<code>bool</code>
<code>x = bytes(5)</code>	<code>bytes</code>
<code>x = bytearray(5)</code>	<code>bytearray</code>
<code>x = memoryview(bytes(5))</code>	<code>memoryview</code>





Python Numbers

- There are **three numeric types** in Python:

`int`

`float`

`complex`

- Variables of numeric types are created when you assign a value to them:

```
x = 1      # int
y = 2.8    # float
z = 1j     # complex
```





Complex Numbers

- Complex numbers are written with a "j" as the imaginary part:

```
x = 3+5j
```

```
y = 5j
```

```
z = -5j
```

```
print(type(x))
```

```
print(type(y))
```

```
print(type(z))
```

Output:

```
<class 'complex'>
```

```
<class 'complex'>
```

```
<class 'complex'>
```





Random Number

- Python has a built-in module called `random` that can be used to make random numbers:

```
import random
```

```
print(random.randrange(1, 10))
```





Python Casting

Integers:

```
x = int(1)      # x will be 1
y = int(2.8)    # y will be 2
z = int("3")    # z will be 3
```

Floats:

```
x = float(1)      # x will be 1.0
y = float(2.8)    # y will be 2.8
z = float("3")    # z will be 3.0
w = float("4.2")  # w will be 4.2
```

Strings:

```
x = str("s1")    # x will be 's1'
y = str(2)        # y will be '2'
z = str(3.0)      # z will be '3.0'
```



Section-2

Python Programming

PYTHON STRING



By: Dr. Imtiaz Ali Korejo



Python Strings

- Strings in python are surrounded by either single quotation marks, or double quotation marks.
- 'hello' is the same as "hello".
- You can display a string literal with the print() function:

```
print("Hello")  
print('Hello')
```





Quotes Inside Quotes

- You can use quotes inside a string, as long as they don't match the quotes surrounding the string:

```
print("It's alright")  
print("He is called 'Johnny'")  
print('He is called "Johnny"')
```





Strings are immutable sequence data type, i.e each time one makes any changes to a string, completely new string object is created

```
a_str = 'Hello World'  
print(a_str)      #output will be whole string. Hello World  
print(a_str[0])   #output will be first character. H  
print(a_str[0:5]) #output will be first five characters. Hello
```





Assign String to a Variable

- Assigning a string to a variable is done with the variable name followed by an equal sign and the string:

```
a = "Hello"  
print(a)
```





Multiline Strings

- You can assign a multiline string to a variable by using three quotes:

```
a = """Lorem ipsum dolor sit amet,  
consectetur adipiscing elit,  
sed do eiusmod tempor incididunt  
ut labore et dolore magna aliqua."""
```

```
print(a)
```





Strings are Arrays

- Like many other popular programming languages, strings in Python are arrays of bytes representing unicode characters.
- However, Python does not have a character data type, a single character is simply a string with a length of 1.
- Square brackets can be used to access elements of the string.

```
a = "Hello, World!"  
print(a[1])
```

Get the character at index 1 (remember that the first character has the index 0):





Looping Through a String

- Since strings are arrays, we can loop through the characters in a string, with a **for** loop.

```
for x in "banana":  
    print(x)
```

Output:

```
b  
a  
n  
a  
n  
a
```





String Length

- To get the length of a string, use the `len()` function.

```
a = "Hello, World!"  
print(len(a))
```

```
Output:  
13
```





Check String

- To check if a certain phrase or character is present **in** a string, we can use the keyword **in**.

Check if "free" is present in the following text:

```
txt = "The best things in life are free!"  
print("free" in txt)
```

Output:
True

Use it in an if statement:

```
txt = "The best things in life are free!"  
if "free" in txt:  
    print("Yes, 'free' is present.")
```

Output:
Yes, 'free' is present.





Check if NOT

- To check if a certain phrase or character is NOT present in a string, we can use the keyword **not in**.

Check if "expensive" is NOT present in the following text:

```
txt = "The best things in life are free!"  
print("expensive" not in txt)
```

Output:
True

Use it in an if statement:

```
txt = "The best things in life are free!"  
if "expensive" not in txt:  
    print("No, 'expensive' is NOT present.")
```

Output:
No, 'expensive' is NOT present.





LEARNING RESOURCES

Source Reference used in Slide

- <https://www.w3schools.com/python>

Other Resources

- <https://realpython.com/>
- <https://docs.python.org/3/>
- <https://docs.python.org/3/tutorial/index.html>
- <https://www.tutorialspoint.com/python/index.htm>



Programming for AI

END of SESSION

Prof. Dr. Imtiaz Ali Korejo

Week-2 , Lecture-4

